DESENVOLVIMENTO FRONT-END · MARCO 4

VueNautas

Gerenciador de tarefas e projetos com Vue.js e TypeScript

Thiago Henrique
Eduarda
Corina



MARCO 4 · ISSUES 70–76

Do requisito ao código

A entrega cobre seis funcionalidades de produto e a apresentação da implementação.

#70 Busca por título

Localiza tarefas por trecho do título.

#71 Filtros combinados

Status e prioridade funcionam juntos.

#72 Estado centralizado

Busca e filtros ficam no Pinia.

#73 Visão em quadro

Agrupa cartões em quatro colunas.

#74 Alternar visualizações

Lista e quadro compartilham os filtros.

#75 Alterar status no cartão

PATCH na API; cartão troca de coluna.

FLUXO DA IMPLEMENTAÇÃO

Componentes Vue
BarraFiltros · CartaoTarefa

→

Estado reativo
Pinia · refs · computed

→

Regras de tela
derivar.ts · agrupamento

→

Persistência
api.ts · PATCH · json-server

Nova lib.
Pinia

Não foi adicionada uma biblioteca para busca, filtros, quadro ou drag-and-drop. Pinia centraliza o estado; Vue Router cuida da navegação.

ISSUE 70 · BUSCA POR TÍTULO

Encontrar por qualquer parte do título

A busca é parcial e não diferencia maiúsculas de minúsculas.

SRC/COMPONENTS/BARRAFILTROS.VUE · CAPTURA DO TEXTO

```
function aoDigitarBusca(event: Event) {
  const alvo = event.target as HTMLInputElement
  store.alterar({ busca: alvo.value })
}
```

SRC/DERIVAR.TS · COMPARAÇÃO DO TEXTO

```
const termo = f.busca.trim()
  .toLocaleLowerCase('pt-BR')
t.titulo.toLocaleLowerCase('pt-BR').includes(termo)
```

COMO O PROCESSO FUNCIONA

- O evento @input captura o que a pessoa digita.
- trim() remove espaços extras no início e no fim.
- toLocaleLowerCase() normaliza o texto; includes() busca um trecho do título.

POR QUE ASSIM?

O usuário não precisa digitar o título completo nem respeitar maiúsculas.

Biblioteca: nenhuma nova. São recursos do JavaScript/TypeScript e reatividade do Vue.

ISSUE 71 · FILTROS DE STATUS E PRIORIDADE

Combinar critérios sem conflito

A tarefa aparece apenas quando atende a todos os filtros ativos.

SRC/DERIVAR.TS · CONDIÇÕES DO FILTRO

```
return tarefas.filter((t) =>
  t.titulo.toLocaleLowerCase('pt-BR').includes(termo) &&
  (f.status === 'todos' || t.status === f.status) &&
  (f.prioridade === 'todas' ||
   t.prioridade === f.prioridade),
)
```

&& significa “E”: título + status + prioridade precisam combinar.

ENTRADA → ESTADO → RESULTADO

1. **Seletores**
   BarraFiltros.vue lê a escolha de status e prioridade.

2. **Atualização**
   store.alterar() registra o critério escolhido.

3. **Filtragem**
   filtrarTarefas() combina os critérios e devolve só as correspondências.

FILTRO NEUTRO

“Todos” e “Todas” significam que aquele critério não restringe resultados.

ISSUE 72 · CENTRALIZAR ESTADO

Uma única fonte para busca e filtros

A tela da lista e a tela do quadro leem e alteram o mesmo store Pinia.

SRC/FILTROSSTORE.TS · STORE COMPARTILHADO

```ts
export const useFiltrosStore = defineStore('filtros', () => {
  const busca = ref('')
  const status = ref<Filtros['status']>('todos')
  const prioridade = ref<Filtros['prioridade']>('todas')
  function alterar(parcial: Partial<Filtros>) { ... }
  function limpar() { ... }
  return { busca, status, prioridade, alterar, limpar }
})
```

ATIVAÇÃO DA BIBLIOTECA

SRC/MAIN.TS

```ts
import { createPinia } from 'pinia'
app.use(createPinia())
```

POR QUE CENTRALIZAR?

- Evita criar cópias independentes dos filtros em cada página.
- O valor escolhido continua ao alternar entre lista e quadro.

Dependência adicionada para esta parte: Pinia. defineStore cria o store; ref cria valores reativos.

ISSUE 73 · VISÃO DE QUADRO

Agrupar tarefas por etapa

O quadro não duplica os dados: ele organiza a mesma lista por status.

SRC/QUADROTAREFAS.VUE · CÁLCULO DAS COLUNAS

```
const colunas = computed(() => {
  const filtradas = filtrarTarefas(
    tarefas.value, storeFiltros)
  return agruparPorStatus(filtradas)
})
```

SRC/DERIVAR.TS · ETAPAS SUPORTADAS

```
export const STATUS: Status[] = [
  'a-fazer', 'em-andamento', 'em-revisao', 'concluida'
]
```

COMO O QUADRO É MONTADO



- computed recalcula quando tarefas ou filtros mudam.
- agruparPorStatus separa as tarefas por cada valor de STATUS.
- v-for desenha as colunas e depois os cartões de cada coluna.

Tecnologia de layout: CSS Grid. Sem biblioteca de Kanban ou arrastar-e-soltar.

Duas visualizações, filtros preservados

Vue Router troca a tela; Pinia mantém o estado dos filtros durante a navegação.

SRC/ROUTER.TS · ROTAS

```
{ path: '/', name: 'tarefas', component: ListaTarefas },

{ path: '/quadro', name: 'quadro', component: QuadroTarefas },
```

SRC/LISTATAREFAS.VUE E SRC/QUADROTAREFAS.VUE · LINKS

```
<RouterLink to="/">Lista</RouterLink>

<RouterLink to="/quadro">Quadro</RouterLink>
```

O QUE ACONTECE AO CLICAR

01 RouterLink navega para a outra rota.

02 O componente da nova visão consulta o store Pinia.

03 Busca e filtros continuam selecionados.

BIBLIOTECAS

Vue Router já gerencia as rotas; Pinia já guarda o estado. Não foi preciso adicionar outra.

Salvar, atualizar a tela e tratar falhas

O seletor dispara uma função; a API confirma a mudança antes de atualizar o cartão.

SRC/COMPOSABLES/USETAREFAS.TS · SALVAR VIA API

```
const atualizada = await atualizarTarefa(id, {
  status: novoStatus,
})
const index = tarefas.value.findIndex(
  (tarefa) => tarefa.id === id,
)
if (index !== -1) tarefas.value[index] = atualizada
```

SRC/COMPONENTS/CARTAOTAREFA.VUE · CONTROLE

```
:disabled="atualizando || !aoMudarStatus"
@change="aoAlterarStatus"
if (!sucesso) statusSelecionado.value = props.tarefa.status
```

PASSO A PASSO DO STATUS

1. O usuário escolhe um dos quatro status no seletor.
2. useTarefas chama atualizarTarefa(), que envia PATCH para /tarefas/:id.
3. Se a API confirma, o item é substituído no array; o quadro recalcula a coluna.
4. Se falhar, a função retorna false, mostra o erro e o seletor volta ao status anterior.

Bibliotecas: nenhuma nova. Vue reativo + composable do projeto + função PATCH já existente em api.ts.

ISSUE 76 · APRESENTAÇÃO E DEMONSTRAÇÃO

Do código ao comportamento visível

DEMONSTRAÇÃO AO VIVO · 2 MINUTOS

01 Digite parte de um título e mostre a busca parcial.

02 Selecione um status e uma prioridade; confirme que os filtros se combinam.

03 Vá de Lista para Quadro e mostre que a busca e os filtros continuam ativos.

04 Mude o status de um cartão e mostre-o na coluna correspondente.

05 Recarregue e confirme que o status permaneceu salvo na API.

PONTOS IMPORTANTES

- Biblioteca adicionada: Pinia, para estado compartilhado.
- Vue Router: navegação entre / e /quadro.
- Sem biblioteca de filtros/Kanban/drag-and-drop.

ARQUITETURA EM UMA FRASE

Vue renderiza os componentes; TypeScript define os tipos e as regras; Pinia guarda busca/filtros; derivar.ts filtra e agrupa; api.ts envia PATCH usando fetch; json-server mantém os dados locais.

CÓDIGO => github.com/thiagopietrobon/por-enquanto-nao-sabemos